

Technical report - Apertus Gate

- **Track:** Track 2B - Apertus Adoption: Own Project
- **Event:** Hack Apertus Online 2026
- **Team:** Unfire - Omar Baró
- **Repository:** <https://github.com/ondmindmanagement-hub/apertus-gate>
- **Demo:** <https://raw.githubusercontent.com/ondmindmanagement-hub/apertus-gate/main/demo/apertus-gate-demo.mp4>

1. Summary

Apertus Gate adds an explicit review boundary before an AI agent performs an external action. A proposed action and its context are sent to Apertus 1.5, which returns a structured risk decision: `allow`, `human_review`, or `required_block`, together with reasons, missing information and a safe next step.

The prototype deliberately does not execute the proposed action. Its job is to create an auditable decision artifact that an existing workflow can enforce. In a live test, Apertus 1.5 70B classified an unapproved EUR 25,000 supplier payment as high risk and `required_block`, directing the workflow to obtain human approval.

2. Purposeful use of Apertus

Apertus is not used as a generic chat layer. It is used specifically at the point where an autonomous system is about to create an external side effect.

The model is asked to reason about:

- reversibility;
- financial, legal and security impact;
- data sensitivity;
- external side effects;
- missing approval or context;
- whether execution should pause for a human.

The output is constrained to a fixed JSON schema:

- `risk_level` : low | medium | high
- `decision` : allow | human_review | required_block
- `reasons`
- `missing_information`
- `safe_next_step`

This makes model reasoning machine-readable and suitable for logging or policy enforcement.

3. Architecture

Browser UI -> Python review service -> OpenAI-compatible Apertus endpoint -> strict JSON normalization -> human-visible decision.

Credentials stay server-side. Inputs are bounded. Invalid or unknown model labels fail toward a safer state: unknown risk becomes `high` and unknown decisions become `human_review`.

Deployment targets

On-premise: the Docker container can run on organisation-managed infrastructure and point to a locally hosted Apertus service.

Air-gapped: the application has no mandatory cloud runtime dependency. The container and Apertus weights can run on an isolated internal network.

Sovereign Swiss cloud: the same application can target a Swiss-hosted Apertus endpoint where jurisdiction and data residency requirements call for it.

The hosted API is a configuration option, not an architectural requirement.

4. Model configuration

- **Default model configuration:** `swiss-ai/apertus-v1.5-8b`
- **Live evidence model:** Apertus 1.5 70B
- **Inference interface:** OpenAI-compatible HTTP endpoint
- **Live evidence infrastructure:** Public AI / CSCS infrastructure in Lugano
- **Fine-tuning:** none
- **Adapters:** none

The system prompt requires the fixed JSON contract and explicitly forbids claiming that an action was executed.

5. Data and evaluation set

No private or human-subject dataset is required.

The repository contains a developer-created evaluation set at `data/evaluation_cases.json`. It currently contains **18 labelled action-boundary scenarios** across low, medium and high risk categories. Examples include public-document summarisation, external email, customer-data transfer, payments, destructive database operations and sensitive-data publication.

The repository also contains:

- `data/demo_benchmark_results.json` - reproducible benchmark output for the deterministic fallback policy;
- `data/apertus_live_run_2026-10-06.json` - a recorded live Apertus model response.

All evaluation prompts were created by the project author for the hackathon. No personal data is included.

6. Technical rigour

The automated test suite now contains **13 tests**, covering:

- JSON extraction from clean model output;
- JSON extraction when surrounding text is present;
- rejection of non-JSON output;
- fail-safe normalization of invalid risk labels;
- fail-safe normalization of invalid decisions;
- safe defaults for missing fields;
- handling of unstructured reasons;
- bounded output lengths;
- representative low-, medium- and high-risk actions;
- full evaluation-dataset consistency.

Current local result:

```
Ran 13 tests
OK
```

The deterministic demo benchmark contains **18 cases and passes 18/18** against its labelled expectations. This benchmark is not presented as a measure of Apertus model quality; it verifies that the offline fallback and submission plumbing are reproducible.

The live Apertus evidence currently contains one high-impact financial case. Apertus 1.5 70B returned:

- risk: `high`
- decision: `required_block`
- safe next step: obtain human approval before execution.

7. Value, cost and scalability

Apertus Gate is intentionally small: it adds one review decision at selected action boundaries rather than placing an LLM in every internal step of an agent workflow.

This creates three practical advantages:

1. **Cost control.** Deployments can review only actions with meaningful external effects. Low-risk internal computation does not need to call the gate.
2. **Model right-sizing.** Apertus 8B can be the default for lower-cost deployments while larger Apertus variants can be reserved for higher-assurance workflows.
3. **Horizontal scalability.** The prototype service is stateless between requests, so multiple instances can sit behind a normal load balancer. Persistent audit storage can be added independently.

The project does not claim a universal cost figure because inference cost depends on deployment, model size and infrastructure. The architecture keeps those choices configurable.

8. Implementation feasibility

The prototype runs with Python 3 and standard-library application code. It is containerised and can be started from a clean checkout.

```
make run
```

Without an API key the interface runs a clearly labelled deterministic demo policy. For real Apertus inference, set `APERTUS_API_KEY` and optionally `APERTUS_API_URL` and `APERTUS_MODEL` .

Tests:

```
make test
```

The design is intentionally separable from any proprietary BOLT implementation. The hackathon repository is a clean-room open-source prototype.

9. Limitations

This is a hackathon prototype, not a formal compliance or safety certification system.

Important limitations:

- LLM decisions can be wrong or inconsistent.
- The live Apertus evidence set is currently small.
- The labelled benchmark is developer-created and requires broader independent validation.
- Production use would need authenticated reviewers, persistent audit storage, organisation-specific policy, rate limiting, stronger input validation, model/version pinning and systematic calibration.
- High-impact workflows should never treat an LLM decision alone as legal or regulatory approval.

10. Next steps

1. Expand live evaluation across the full 18-case benchmark.
2. Compare Apertus 8B and 70B for consistency, latency and resource cost.
3. Add signed decision records and tamper-evident audit logs.
4. Add an authenticated human approval queue.
5. Add policy profiles for finance, software deployment, data access and procurement.
6. Measure end-to-end latency and compute cost per reviewed action.

11. Reproducibility

Repository: <https://github.com/ondmindmanagement-hub/apertus-gate>

Code and evaluation artifacts are stored together so the jury can inspect the implementation and reproduce the local tests.

License

Source code: Apache License 2.0.

Documentation and non-code text: Creative Commons Attribution 4.0 (CC-BY-4.0).

References

- Apertus documentation: <https://apertus-ai.org/docs/>
- Hack Apertus Track 2B template: https://github.com/HackApertus/project-template/tree/main/track_2b
- Public AI API documentation: <https://platform.publicai.co/docs>

[executed on device: MacBook-Pro-de-Omar.local (21fdbe82-58ff-459f-821a-67c612bf8202)]